

Rhinestone Smart Sessions V2 Security Audit

: Rhinestone

Jan 19, 2026

Revision 1.0

ChainLight@Theori

Theori, Inc. (“We”) is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

Table of Contents

Rhinestone Smart Sessions V2 Security Audit	1
Table of Contents	2
Executive Summary	4
Audit Overview	5
Scope	5
Code Revision	5
Severity Categories	6
Status Categories	8
Finding Breakdown by Severity	9
Findings	10
Summary	10
#1 SmartSessionsV2-001 BaseClaimPolicy.initializeWithMultiplexer() should overwrite all previous values on re-invocation	11
#2 SmartSessionsV2-002 Incorrect data slicing in SmartSessionEmissary leading to verifyClaim revert	14
#3 SmartSessionsV2-003 The requestSender address changes during nested check1271SignedAction() calls	15
#4 SmartSessionsV2-004 Although SmartSessionManager allows a single permissionId to manage multiple lockTags, conflicts may occur when calling setConfig() and removeConfig()	18
#5 SmartSessionsV2-005 Minor Suggestions	20
Revision History	23

Executive Summary

Beginning on December 15, 2025, ChainLight completed the first-phase audit, concluding on December 29, 2025, and additionally completed a second-phase audit of Pull Request #39 in the smart-sessions-v2 repository from January 5, 2026 through January 8, 2026. The audit focused on validating the safety and correctness of the claim validation and policy initialization flows, with particular emphasis on access control and one-time initialization guarantees for `initializeWithMultiplexer()`, signature verification correctness across Emissary modes, consistent `requestSender` propagation during nested sub-policy validation, and the alignment of encoding/layout assumptions between code, comments, and documentation.

The audit revealed a total of 5 issues, categorized by severity as follows:

- Total: 5
- Medium: 3
- Low: 1
- Informational: 1

The commit where all recommendations have been patched is [https://github.com/rhinestonewtf/smart-sessions-v2:5a1463a7ac5b1b0d4b775dc69106ae9c0dc85e23](https://github.com/rhinestonewtf/smart-sessions-v2/commit/5a1463a7ac5b1b0d4b775dc69106ae9c0dc85e23).

Audit Overview

Scope

Name	Rhinestone Smart Sessions V2 Security Audit
Target / Version	• https://github.com/rhinestonewtf/smart-sessions-v2: aedd51f3444fd2e7acd524bde5183d8678cb9ef9 • https://github.com/rhinestonewtf/smart-sessions-v2/pull/39: 02c22ccf06842f40fa3d6133df63dacbca8c9016
Application Type	Smart contracts
Lang. / Platforms	Smart contracts [Solidity]

Code Revision

N/A

Severity Categories

Severity	Description
Critical	The attack cost is low (not requiring much time or effort to succeed in the actual attack), and the vulnerability causes a high-impact issue. (e.g., Effect on service availability, Attacker taking financial gain)
High	An attacker can succeed in an attack which clearly causes problems in the service's operation. Even when the attack cost is high, the severity of the issue is considered "high" if the impact of the attack is remarkably high.
Medium	An attacker may perform an unintended action in the service, and the action may impact service operation. However, there are some restrictions for the actual attack to succeed.
Low	An attacker can perform an unintended action in the service, but the action does not cause significant impact or the success rate of the attack is remarkably low.
Informational	Any informational findings that do not directly impact the user or the protocol.
Note	Neutral information about the target that is not directly related to the project's safety and security.

Status Categories

Status	Description
Reported	ChainLight reported the issue to the client.
WIP	The client is working on the patch.
Patched	The client fully resolved the issue by patching the root cause.
Mitigated	The client resolved the issue by reducing the risk to an acceptable level by introducing mitigations.
Acknowledged	The client acknowledged the potential risk, but they will resolve it later.
Won't Fix	The client acknowledged the potential risk, but they decided to accept the risk.

Finding Breakdown by Severity

Category	Count	Findings
Critical	0	N/A
High	0	N/A
Medium	3	- SmartSessionsV2-001 - SmartSessionsV2-002 - SmartSessionsV2-004
Low	1	SmartSessionsV2-003
Informational	1	SmartSessionsV2-005
Note	0	N/A

Findings

Summary

#	ID	Title	Severity	Status
1	SmartSessionsV2-001	<code>BaseClaimPolicy.initializeWithMultiplexer()</code> should overwrite all previous values on re-invocation	Medium	Patched
2	SmartSessionsV2-002	Incorrect data slicing in <code>SmartSessionEmissary</code> leading to <code>verifyClaim</code> revert	Medium	Patched
3	SmartSessionsV2-003	The <code>requestSender</code> address changes during nested <code>check1271SignedAction()</code> calls	Low	Patched
4	SmartSessionsV2-004	Although <code>SmartSessionManager</code> allows a single <code>permissionId</code> to manage multiple <code>lockTags</code> , conflicts may occur when calling <code>setConfig()</code> and <code>removeConfig()</code>	Medium	Patched
5	SmartSessionsV2-005	Minor Suggestions	Informational	Patched

#1 SmartSessionsV2-001

BaseClaimPolicy.initializeWithMultiplexer() should
overwrite all previous values on re-invocation

ID	Summary	Severity
SmartSessionsV2-001	The <code>BaseClaimPolicy.initializeWithMultiplexer()</code> function does not overwrite previous configuration values, violating the <code>IPolicy</code> interface specification. This lack of reset logic allows stale data, such as arbiter sets and recipient mappings, to persist and merge with new configurations.	Medium

Description

According to the `IPolicy.initializeWithMultiplexer` comments, the initialization process must overwrite all prior values to ensure a clean state. However, the current implementation of `BaseClaimPolicy` lacks explicit reset logic. Consequently, when the function is called, existing elements in sets (e.g. from `initializeArbiter()`) are not cleared, and mappings keyed by `chainId` (e.g. from `initializeRecipient()`) are not deleted. This results in new configurations being appended to or merged with the existing state rather than replacing it.

Impact

Medium

Not clearing previous state variables leads to the retention of residual configuration data. This creates an inconsistency where revoked arbiters or obsolete recipient details remain active alongside new values, potentially enforcing unintended or insecure policy restrictions.

Recommendation

Implement explicit reset logic within `initializeWithMultiplexer()` to clear all relevant state variables before applying new configurations. Specifically, ensure that all elements in sets are removed and that mapping entries corresponding to existing keys are deleted.

Remediation

Patched

`initializeWithMultiplexer()` was patched to be one-time only per `BasePolicyStorage`, preventing repeated initialization on the same storage instance.

#2 SmartSessionsV2-002 Incorrect data slicing in SmartSessionEmissary leading to verifyClaim revert

ID	Summary	Severity
SmartSessionsV2-002	In EMISSARY_VANILLA mode, the SmartSessionEmissary.verifyClaim() function incorrectly passes the full emissaryData to the signature validator. This malformed input causes the verifyClaim function to revert, rendering the mode unusable for valid claims.	Medium

Description

The SmartSessionEmissary.verifyClaim() function is responsible for validating claims based on the configured mode. Currently, when the mode is EMISSARY_VANILLA, the system calls _validateSignature() using the entire emissaryData instead of the required emissaryData[1:]. This data format mismatch causes the internal validation logic to fail and the verifyClaim transaction to revert.

Impact

Medium

The incorrect slicing guarantees that verifyClaim will revert for every attempt to use EMISSARY_VANILLA mode. This results in a complete denial of service for this functionality, preventing the processing of any legitimate sessions or transactions.

Recommendation

Update the verifyClaim() function to pass emissaryData[1:] (the data excluding the first byte) to _validateSignature() when the mode is set to EMISSARY_VANILLA.

Remediation

Patched

The issue has been resolved as recommended.

#3 SmartSessionsV2-003 The requestSender address changes during nested check1271SignedAction() calls

ID	Summary	Severity
SmartSessionsV2-003	A mismatch occurs between the original requestSender provided to BaseClaimPolicy.check1271SignedAction() and the requestSender value propagated to sub-policies during nested check1271SignedAction() calls.	Low

Description

When BaseClaimPolicy.check1271SignedAction() is invoked, the omitted second parameter, requestSender, is the same as msg.sender when SmartSessionEmissary.verifyClaim() is called. However, when BaseClaimPolicy.check1271SignedAction() calls a sub-policy's check1271SignedAction() (via BaseValidationLib's _validate_____subPolicy functions), it uses msg.sender as requestSender. In this context, msg.sender becomes the SmartSessionEmissary contract address, which differs from the original requestSender used when BaseClaimPolicy.check1271SignedAction() was first called.

Impact

Low

During nested validation, sub-policies may receive a different requestSender (the SmartSessionEmissary address) than the original requestSender used by the parent BaseClaimPolicy, leading to inconsistent validation and potentially unintended allow/deny outcomes when requestSender is used in checks.

Recommendation

The original requestSender from the initial BaseClaimPolicy.check1271SignedAction() call should be forwarded unchanged as the requestSender argument to subPolicy.check1271SignedAction().

Remediation

Patched

The implementation was patched to pass `address(0)` as `requestSender` when invoking a sub-policy's `check1271SignedAction()`, explicitly signaling the absence of an external `requestSender` and eliminating ambiguity within sub-policies.

#4 SmartSessionsV2-004 Although SmartSessionManager allows a single `permissionId` to manage multiple `lockTag`s, conflicts may occur when calling `setConfig()` and `removeConfig()`

ID	Summary	Severity
SmartSessionsV2-004	SmartSessionManager allows a single <code>permissionId</code> (session) to manage multiple <code>lockTags</code> , but enabling/disabling policies can overwrite or delete state in ways that are incompatible with multi- <code>lockTag</code> usage.	Medium

Description

When a single `permissionId` is reused across multiple `lockTags`:

- Policy overwrite during enable: If `_enableSession()` is called multiple times for different `lockTags` using the same `permissionId`, the policies for multiple `lockTags` accumulate within `$enabledERC7739`, `$erc1271Policies`, and `$actionPolicies`. This can lead to the simultaneous existence of conflicting policies (e.g., policies with overlapping or differing Time Bounds). Furthermore, if a policy already exists in `$erc1271Policies` or `$actionPolicies`, the call to `initializeWithMultiplexer()` resets the previous configuration values, resulting in only the settings for the most recent `lockTag` being reflected.
- Over-deletion during disable: It is not possible to remove configuration for only a single `lockTag`. Even if `_disablePolicies()` is invoked to remove just one `lockTag`, it deletes `$erc1271Policies`, `$actionPolicies`, `$enabledERC7739`, `$sessionValidators`, and `$enabledSessions`, preventing the same `permissionId` (session) from being reused for other `lockTags`.

Impact

Medium

Multi-`lockTag` sessions can end up with inconsistent or unintended policy configuration, and disabling a single `lockTag` can unintentionally disable the entire session.

Recommendation

If a single `permissionId` is intended to allow multiple `lockTags` , the following changes are recommended:

- Add `lockTag` as part of the key for `$enabledERC7739` , `$erc1271Policies` , and `$actionPolicies` .
- Implement separate functions for (1) removing all `lockTags` under a session (`permissionId`) and (2) removing an individual `lockTag` .

Alternatively, restrict each `permissionId` to allow only a single `lockTag` to avoid these conflicts entirely.

Remediation

Patched

It was updated so that each `permissionId` can have only a single `lockTag` .

#5 SmartSessionsV2-005 Minor Suggestions

ID	Summary	Severity
SmartSessionsV2-005	The description includes multiple suggestions for preventing incorrect settings caused by operational mistakes, mitigating potential issues, improving code maturity and readability, and other minor issues.	Informational

Description

1. Contrary to what the README states, `ModelLib.sol` defines the 1271 / 7739 signature mode flags differently: `IS_VALID_SIG_1271` is `0x00` (not `0x02`) and `IS_VALID_SIG_1271_7739` is `0x01` (not `0x03`). Either the README should be updated or the constant values should be changed to match the documentation.
2. In `BaseConfigLib.initializeExpiry()`, the comment describing the Expiry layout states that `min` is `packed[0:16]` and `max` is `packed[16:32]`, but `unpackUint128()` returns values in the opposite order (`packed[16:32]`, then `packed[0:16]`), which causes `min` and `max` to be parsed in reverse. This should be corrected by using `(uint128 max, uint128 min) = unpackUint128(packed);`, or alternatively by updating the layout comment to `[maxExpiry: 16 bytes][minExpiry: 16 bytes]`. The same issue applies to `initializeFillExpiry()`.
3. During a Compact claim, it is unclear whether the value stored in `tokenInSet` is encoded as `token + lockTag` or `lockTag + token`. Multiple call sites and comments describe it as `token + lockTag` (e.g., `BaseStorageLib`, `CompactClaimPolicy`, and `CompactValidation`), while the comment on `CompactConfigLib.initializeTokenIn()` describes it as `lockTag + token`, so the implementation and documentation should be aligned.
4. In `SmartSessionMixin`, the comment in `_claimIsValidSignatureNowCallldata()` implies that `lockTag` is explicitly validated, but the current logic does not directly enforce this at that point. We recommend adding an explicit `lockTag` validation to match the comment (e.g., checking both the session and `$enabledLockTags[permissionId]`), even though an unregistered `lockTag` would still revert later in `$claimPolicies[lockTag].checkERC1271()`. In addition, the comment in `_enforceActionPolicies()` should remove the “LockTag” wording if it is not actually enforced there.
5. In `Permit2ClaimPolicy.getTokenInWhitelist()` and `isTokenInWhitelisted()`, the comment stating that `chainId` is “typically 0 for origin chain” is incorrect: `chainId = 0` is used as a

wildcard when the mode is `MODE_CHECK_CATCHALL` , so the comment should be updated to reflect the wildcard semantics.

6. In `SmartSessionMixin._claimIsValidSignatureNowCalldata()` , the comment describing `sigLength` is incorrect and should instead refer to `policyDataOffset` , so the comment should be updated to match the actual variable meaning and usage.

7. Contrary to the comment in

`SmartSessionManager._useTransientReentrancyGuardOnlyOnMainnet()` (“Always use transient reentrancy guard only on mainnet”), if the function returns `false` , the transient reentrancy guard is used unconditionally, regardless of whether the deployment is on mainnet. We recommend either updating the comments or changing `_useTransientReentrancyGuardOnlyOnMainnet()` to return `true` instead of `false` .

Impact

Informational

Recommendation

Consider applying the suggestions in the description above.

Remediation

Patched

It has been patched as recommended.

Revision History

Version	Date	Description
1.0	Jan 19, 2026	Initial version
1.1	Jan 20, 2026	Added post-patch commits

Theori, Inc. (“We”) is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

